End-to-End Training Time Analysis

0.0.1 End-to-End Training Time

We measure end-to-end training time as the total simulated wall-clock time reported by each experiment, which includes local training, encryption, server-side aggregation, and ciphertext transmission. This is distinct from the pure serial communication latency (`TT`) used in Table 1, which only counts $\sum \max_i t_{\text{up}}^{(i)} + \sum \max_i t_{\text{down}}^{(i)}$.

Overall comparison. Figure 1 compares the total training time across the four datasets. On MNIST, FMNIST, and CIFAR-10, the non-prefetch baselines record the shortest end-to-end times; FedPHE is fastest on all three (202 s, 179 s, and 1221 s), followed by ESAFL and FedAvg+CKKS. ECPFL-RK requires 1.5×–3.3× more wall-clock time than FedAvg+CKKS on these three datasets, because top-k sparsification, client-side encryption, and CipherFetch prefetching introduce additional per-round overhead. On SVHN, however, ECPFL-RK finishes 467 s earlier than FedAvg+CKKS (1.6× faster). MASER is the clear outlier: its multi-key CKKS partial-decryption protocol makes it 42.8×–102.7× slower than FedAvg+CKKS (60× on average), reaching up to 1.3×10^5 s on SVHN.

Communication bottleneck. To understand how much of the wall-clock cost is pure network transfer, Figure 2 reports the serial communication time ($\sum \max_i t_{\text{up}}^{(i)} + \sum \max_i t_{\text{down}}^{(i)}$). ECPFL-RK reduces this metric relative to FedAvg+CKKS by 1.8×–5.7× (3.9× on average), cutting MNIST/FMNIST transfers from 452/411 s to 80/73 s and the SVHN transfer from 1930 s to 801 s. ECPFL-TH obtains comparable or larger reductions on CIFAR-10 and SVHN (4.5× and 3.9×, respectively), although its more aggressive thresholding is less effective on the smaller datasets. MASER's pure network time is not the dominant factor (463–3956 s); the orders-of-magnitude gap in Figure 1 therefore stems from unmeasured server-side multi-key CKKS aggregation and partial-decryption overhead, which we subsume under the residual wall-clock term.

Compute–communication trade-off. Figure 3 places every (method, dataset) pair in the compute–communication plane. The diagonal dashed line marks equal compute and communication time. FedAvg+CKKS, FedPHE, and ESAFL cluster in the upper-right region, where serial communication exceeds local computation; this confirms that the network is their primary bottleneck. ECPFL variants shift leftward and slightly upward: their serial communication drops sharply, while their compute time increases modestly (average 188 s for ECPFL-RK versus 103 s for FedAvg+CKKS) because of additional encryption and prefetch operations. MASER lies off the diagonal in a different way: its measured compute and communication are comparable to the baselines, yet its end-to-end time is one to two orders of magnitude larger, again indicating that the dominant cost is server-side cryptographic overhead not captured by the per-client compute components.

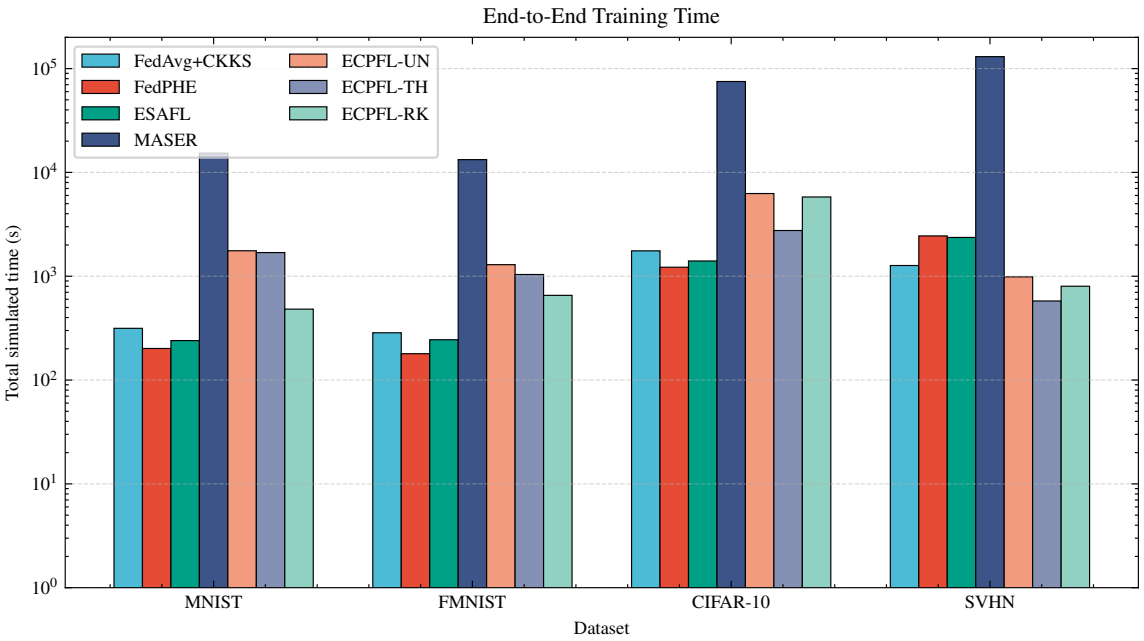


Figure 1: End-to-end training time across datasets and methods. Values are total simulated wall-clock times over 50 rounds.

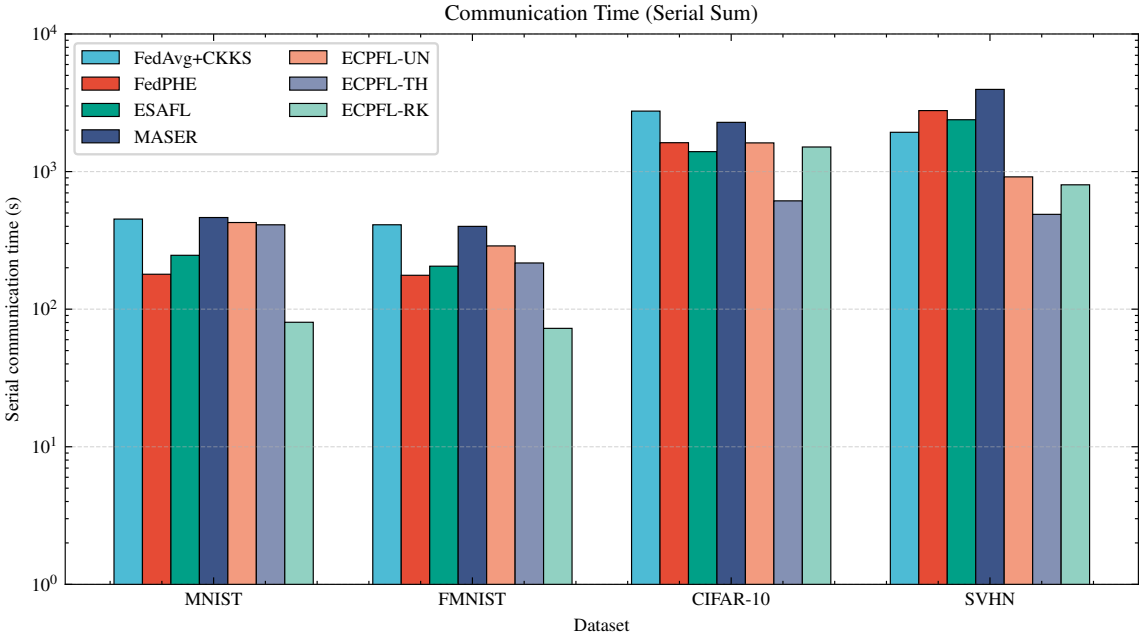


Figure 2: Serial communication time across datasets and methods.

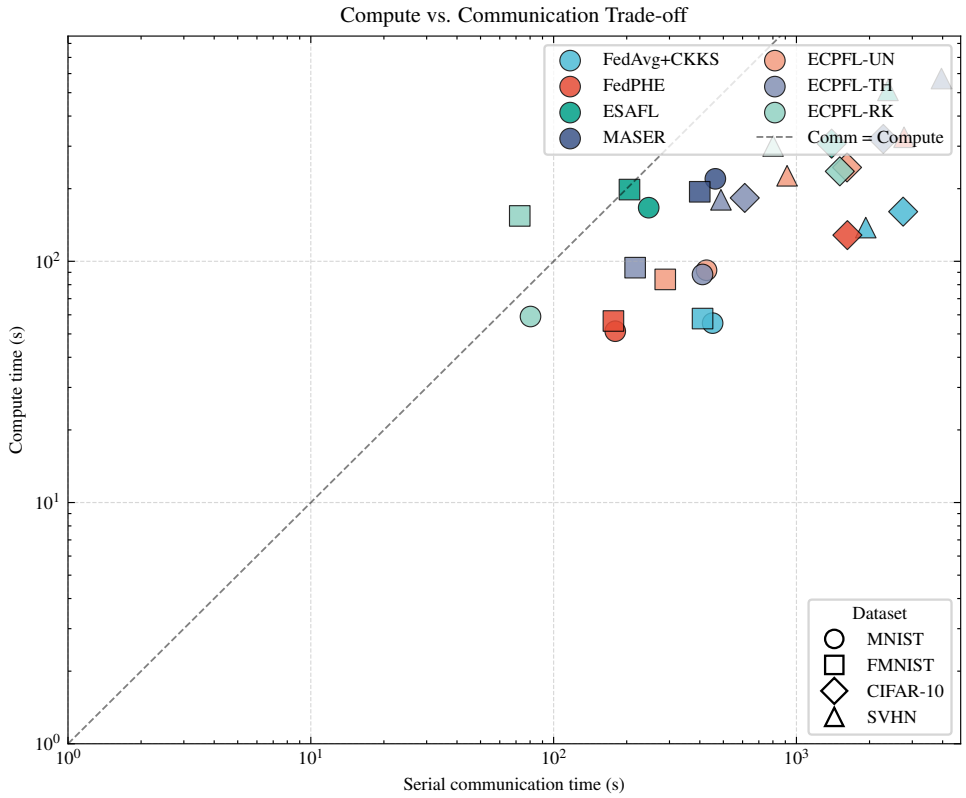


Figure 3: Compute versus serial communication time. Each point corresponds to one method–dataset pair. The dashed line represents equal compute and communication time.